

Hixtory

An adaptive history and geography game for PSAC students

Group report and division of work

Member	Name	Student ID	Part
Member 1	Shadoo-buccus Ahmad Ajmeer	2416162	Part 1
Member 2	Sahabun Hishaam	2414558	Part 2
Member 3	Tilagovindoo Thayalan	2414604	Part 3

Lecturer: Pudaruth Sameerchand

University of Mauritius, Year 3

Date: 24 September 2026

Contents

Executive summary	3
1. Introduction	3
1.1 Background	3
1.2 Aim and objectives	3
1.3 Scope	3
2. Background	4
2.1 Gamified learning and retrieval practice	4
2.2 Adaptive difficulty	4
2.3 The AI perspective	4
2.4 Technology	4
3. System overview	4
3.1 Game design	4
3.2 Architecture	5
3.3 Adaptive engine	5
4. Division of work	6
4.1 Summary	6
4.2 Part 1: Educational content and game design (Member 1)	6
4.3 Part 2: React front-end development (Member 2)	7
4.4 Part 3: Adaptive AI engine, testing and evaluation (Member 3)	8
4.5 Shared responsibilities	8
5. Collaboration and integration	8
6. Timeline	9
7. Risks and mitigation	9
8. Testing and evaluation plan	10
8.1 Unit and functional testing	10
8.2 Content verification	10
8.3 Usability and accessibility	10
8.4 Evaluation of the adaptive engine	10
8.5 Privacy and security	11
9. Conclusion	11
References	12

Executive summary

This report describes Hixtory, a browser-based game built with React that teaches Mauritian history and geography to pupils preparing for the Primary School Achievement Certificate (PSAC). Players answer multiple-choice questions to earn points and unlock four stages. An adaptive engine estimates the player's level in each subject and selects questions of matching difficulty. This is the artificial intelligence component of the project.

The work is divided among three members. Part 1 covers educational content and game design. Part 2 covers the React front end. Part 3 covers the adaptive AI engine, testing and evaluation. The report sets out each member's responsibilities, deliverables and acceptance criteria, together with the integration approach, timeline, risks and evaluation plan.

1. Introduction

1.1 Background

The PSAC is the end-of-primary examination in Mauritius, and History and Geography is one of the subjects pupils revise. Revision often relies on textbooks and past papers, which give little immediate feedback. A game that asks questions, explains each answer and rewards progress can make revision more engaging. Gamification, the use of game design elements in non-game contexts (Deterding et al., 2011), is widely used for this purpose.

1.2 Aim and objectives

The aim is to design and build a web game that teaches history and geography to PSAC pupils, in which the player answers questions to earn points and progress through stages, and in which AI is used while the game runs. The objectives are to:

- build a question bank aligned with the PSAC History and Geography content;
- implement a stage-based game in React with points, hints and stage unlocking;
- implement an adaptive engine that matches question difficulty to each player;
- test the game for correctness, usability and accessibility; and
- divide the work fairly among the three group members.

1.3 Scope

The current prototype has four stages, multiple-choice questions and a single player. It runs in the browser, needs no login and stores no personal data. Extensions such as an AI tutor are discussed in Section 4.4 but are not required for the core game.

2. Background

2.1 Gamified learning and retrieval practice

Reviews of empirical studies report that gamification often has positive effects, although the results depend on the context and on the users (Hamari, Koivisto and Sarsa, 2014). A question-and-answer game also fits well with retrieval practice. Roediger and Karpicke (2006) found that taking tests improved long-term retention compared with simply re-reading material. Giving an explanation after every answer, as Hixtory does, turns each question into a small learning moment.

2.2 Adaptive difficulty

Vygotsky (1978) described the zone of proximal development, the range of tasks a learner can complete with support but not yet alone. Csikszentmihalyi (1990) described flow as a state that occurs when the challenge of a task is balanced with the person's skill. Both ideas suggest that questions should be neither too easy nor too hard. An adaptive game can achieve this by tracking how well the player is doing and choosing the next question accordingly.

2.3 The AI perspective

Russell and Norvig (2021) describe an intelligent agent as a system that perceives its environment and chooses actions to achieve a goal. In Hixtory, the adaptive engine can be viewed as a simple agent. It observes the player's answers, updates a model of the player's skill and chooses the next question so that the difficulty stays appropriate. The approach is inspired by knowledge tracing, which estimates a student's mastery from the sequence of correct and incorrect answers (Corbett and Anderson, 1994). Hixtory uses a simplified rule-based update rather than the full probabilistic model, which keeps it easy to explain and to test.

2.4 Technology

The game is built with React, a JavaScript library for building user interfaces from reusable components that re-render when their state changes (React, no date). The project is created and served with Vite, a modern front-end build tool (Vite, no date).

3. System overview

3.1 Game design

Hixtory takes the player on a journey through the story and map of Mauritius. Each stage has its own theme and uses one of the four colours of the Mauritian flag. Table 1 lists the stages.

Table 1: Stages of the game

Stage	Theme	Subject	Colour
1	Dutch and French times	History	Red
2	British rule and labour	History	Blue

Stage	Theme	Subject	Colour
3	Independence	History	Yellow
4	Map of Mauritius	Geography	Green

The rules of the prototype are as follows:

- Each stage asks five questions, chosen from a bank of six per stage.
- A correct answer earns 10, 20 or 30 points depending on whether the question is easy, medium or hard.
- The player can use one hint per question, which removes two wrong answers but halves the points.
- After each answer the game shows whether it was right and a short explanation.
- The player needs at least three correct answers to unlock the next stage.

3.2 Architecture

The prototype is a single-page React application. Game state (screen, stage, score, used questions and skill estimates) is held in one reducer, so every change to the game follows a clear, testable path. Table 2 lists the main files.

Table 2: Main files of the prototype

File	Purpose
src/data/questions.js	Stage definitions and the question bank (stage, level, question, options, answer and explanation).
src/adaptive.js	The adaptive engine: skill update and next-question selection.
src/App.jsx	The user interface and the game reducer.
src/index.css	Styles based on the colours of the Mauritian flag.

3.3 Adaptive engine

The engine keeps one skill estimate per subject, on the same scale as the question levels (1 = easy, 2 = medium, 3 = hard). Every player starts at 2. After each answer the estimate moves up or down, and the next question is the unused one in the stage whose level is closest to the estimate. Table 3 shows the current settings.

Table 3: Settings of the adaptive engine

Setting	Value	Meaning
Starting skill	2.0	Every player begins at medium difficulty.
Correct answer	+0.35	The estimate rises after a correct answer.

Setting	Value	Meaning
Correct with hint	+0.15	A hint suggests the pupil was less sure, so the rise is smaller.
Wrong answer	-0.50	The estimate falls faster than it rises, so pupils are not pushed into questions that are too hard.
Range	1 to 3	The estimate is limited to the range of question levels.

These values are starting points. Part 3 will tune them using simulated learners and pilot testing (Section 8).

4. Division of work

4.1 Summary

The project is divided into three parts of similar size. Each part has a clear owner, so the members can work in parallel. Table 4 gives an overview.

Table 4: Overview of the three parts

Part	Member	Focus	Main deliverables
1	Ajmeer	Educational content and game design	Verified question bank, game design document, storyboard, scoring rules
2	Hishaam	React front-end development	Working user interface, component structure, accessible and responsive layout, deployment
3	Thayalan	Adaptive AI engine, testing and evaluation	Adaptive engine, unit tests, learner simulations, pilot test results

4.2 Part 1: Educational content and game design (Ajmeer)

Ajmeer is responsible for what the pupils learn and how the game rewards them. The quality of the questions decides whether the game is useful, so this part also owns the accuracy of the content.

Responsibilities

- Study the PSAC History and Geography content and decide which topics each stage covers.
- Extend the question bank from 24 to at least 60 questions, with a balance of easy, medium and hard questions in every stage.
- Write a short, pupil-friendly explanation for every answer.

- Check every fact against the PSAC textbook and an authoritative source, and record the source.
- Define the scoring, hint and stage-unlocking rules, and produce the storyboard with Member 2.
- Write Sections 1 and 2 of the report and edit the final reference list.

Deliverables

- An updated `src/data/questions.js` that follows the agreed data format.
- A content verification log listing the source for each question.
- A game design document and a storyboard of the screens.

Done when

- Every question has a recorded source and has been checked by a second member.
- Each stage has at least two questions at every difficulty level, so the adaptive engine always has a choice.
- The explanations are short and written at a level suitable for a primary school pupil.

4.3 Part 2: React front-end development (Hishaam)

Hishaam is responsible for the software that pupils see and use, from the project setup to the final build.

Responsibilities

- Create the Git repository and the Vite and React project, and write the setup instructions.
- Split the interface into components (start screen, stage map, question card, feedback and result screen) and keep the game state in the reducer.
- Style the game with the flag-based colours, clear typography and large touch-friendly buttons.
- Make the layout responsive, keyboard accessible and readable, following the accessibility guidelines (W3C, 2018) and the usability heuristics (Nielsen, 1994).
- Save the player's progress in the browser so pupils can continue later.
- Produce the production build and host it, and write Section 3 of the report.

Deliverables

- A working, deployed application and a README with run instructions.
- A component structure that Members 1 and 3 can work with without editing each other's files.
- Screenshots for the report.

Done when

- The game works on a screen 360 pixels wide and on a desktop browser.
- All actions can be completed with the keyboard, and text meets the minimum contrast ratio of 4.5:1.

- There are no errors in the browser console during a full play-through.

4.4 Part 3: Adaptive AI engine, testing and evaluation (Thayalan)

Thayalan is responsible for the AI component and for showing that it works, which is the part of the project that most directly answers the module's learning outcomes.

Responsibilities

- Own src/adaptive.js: the skill model, the update rule and the question selection.
- Tune the settings in Table 3, for example the starting level and the step sizes.
- Add a summary at the end of each stage that tells the pupil which subject to revise.
- Write unit tests for the engine, for example with Vitest.
- Simulate learners of different abilities and measure whether the engine serves them appropriate questions.
- Plan and run a small pilot test, and write Sections 5 and 8 of the report together with the discussion of limitations.

Deliverables

- A tested adaptive.js with documented function signatures.
- Simulation results (tables or charts) and the pilot test findings.
- An evaluation section that explains the method, the results and the limitations.

Done when

- The skill estimate always stays between 1 and 3 and no question is repeated within a stage.
- In simulation, stronger learners receive harder questions on average than weaker learners.
- All unit tests pass.

4.5 Shared responsibilities

- All members attend a weekly meeting and record decisions in a shared log.
- Each member reviews at least one pull request from another member every week.
- Each member writes the report sections for their own part and proofreads one other part.
- All members check that every in-text citation appears in the reference list, and the reverse.

5. Collaboration and integration

The three parts depend on each other, so the group agrees two interfaces at the start. The first is the question format: each question has an id, a stage, a level from 1 to 3, the question text, an array of options, the index of the correct answer and an explanation. The second is the function signatures in adaptive.js: `updateSkill(skill, correct, usedHint)` returns the new skill estimate, and `pickNext(pool, usedIds, skill)` returns the next question. Once these are fixed, each member can work without waiting for the others.

The code is kept in a shared Git repository. Each member works on a separate branch, and changes reach the main branch only through a pull request that another member has reviewed (Chacon and Straub, 2014). The group works in weekly increments, integrating and testing the whole game at the end of each week, which reduces the risk of large integration problems late in the project (Sommerville, 2016).

6. Timeline

Table 5 shows a four-week plan. The weeks should be adjusted to match the submission deadline.

Table 5: Four-week plan

Week	Member 1: content and design	Member 2: front end	Member 3: AI and evaluation
1	Agree the curriculum scope, write the design document and draft 30 questions.	Set up the repository and project, agree the component structure and build the start screen and stage map.	Agree the interfaces, refine the adaptive design and write the test plan.
2	Complete 60 questions with explanations and sources.	Build the question, feedback and result screens and connect them to the game state.	Implement and tune the adaptive engine and add the revision summary.
3	Have a second member check the facts and write the background sections.	Add accessibility, responsive layout and saved progress.	Run the unit tests and learner simulations and prepare the pilot test.
4	Proofread the report and finalise the references.	Build, deploy, write the README and take screenshots.	Report the pilot results and write the evaluation and limitations.

7. Risks and mitigation

Table 6: Main risks

Risk	Effect	Mitigation
Factual errors in questions	Pupils learn wrong information and the game loses credibility.	Record a source for every question and have a second member check it.
Merge conflicts between members	Lost work and delays.	Use separate branches, small pull requests and one owner per file.

Risk	Effect	Mitigation
API key exposed in the browser	Anyone could misuse the key and its cost.	Keep keys on a server, or do not use an external AI service in the submitted version (OWASP, no date).
Adaptive settings are poorly tuned	Questions feel too easy or too hard.	Test with simulated learners and adjust the settings in Table 3.
Scope creep	Core features are unfinished at the deadline.	Finish the core game first and treat the AI tutor as optional.
A member is unavailable	Their part is delayed.	Keep the shared log up to date so another member can take over.

8. Testing and evaluation plan

8.1 Unit and functional testing

Member 3 will write automated tests for `updateSkill` and `pickNext`, including the boundary cases where the skill estimate reaches 1 or 3 and the case where only one unused question is left. Member 2 will test a full play-through of every stage, including a failed stage and a retry.

8.2 Content verification

Every question will be checked by two members. This step catches subtle problems. For example, the UNESCO World Heritage Centre records that indentured labourers began arriving in 1834 but that the immigration depot at Aapravasi Ghat was built in 1849, so the wording of any question on this topic needs care.

8.3 Usability and accessibility

The interface will be reviewed against the usability heuristics, in particular visibility of system status and error prevention (Nielsen, 1994), and against the contrast and keyboard requirements of the Web Content Accessibility Guidelines (W3C, 2018).

8.4 Evaluation of the adaptive engine

Virtual learners with fixed probabilities of answering easy, medium and hard questions correctly will play thousands of simulated stages. The group will compare the average difficulty served to strong and weak learners, and how quickly the skill estimate settles. A small pilot with a few pupils or teachers will follow, with permission from a teacher or guardian, to check that the difficulty feels right and the explanations are clear. The results will be reported honestly, including the limits of a small sample.

8.5 Privacy and security

The game stores no names, accounts or other personal data, and saved progress stays in the pupil's own browser. If an external AI service is added, requests will pass through a server that holds the key.

9. Conclusion

Hixtory combines a curriculum-based question game with an adaptive engine that adjusts difficulty to each pupil. Dividing the project into content and design, front-end development, and AI and evaluation gives each member a substantial and distinct contribution, with agreed interfaces that allow the parts to be developed in parallel. The evaluation plan will show whether the adaptive engine does what it is designed to do, and the group will report its limitations openly.

References

- Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. Berkeley, CA: Apress. Available at: <https://git-scm.com/book/en/v2> (Accessed: 24 September 2026).
- Corbett, A.T. and Anderson, J.R. (1994) 'Knowledge tracing: modeling the acquisition of procedural knowledge', *User Modeling and User-Adapted Interaction*, 4(4), pp. 253–278. doi: 10.1007/BF01099821.
- Csikszentmihalyi, M. (1990) *Flow: the psychology of optimal experience*. New York: Harper & Row.
- Deterding, S., Dixon, D., Khaled, R. and Nacke, L. (2011) 'From game design elements to gamefulness: defining "gamification"', in *Proceedings of the 15th International Academic MindTrek Conference*. Tampere, Finland, 28–30 September. New York: ACM, pp. 9–15. doi: 10.1145/2181037.2181040.
- Hamari, J., Koivisto, J. and Sarsa, H. (2014) 'Does gamification work? A literature review of empirical studies on gamification', in *Proceedings of the 47th Hawaii International Conference on System Sciences*. Waikoloa, HI, 6–9 January. Los Alamitos, CA: IEEE, pp. 3025–3034. doi: 10.1109/HICSS.2014.377.
- Nielsen, J. (1994) *10 usability heuristics for user interface design*. Nielsen Norman Group. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/> (Accessed: 24 September 2026).
- OWASP (no date) *OWASP Top Ten*. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 24 September 2026).
- React (no date) *React documentation*. Available at: <https://react.dev> (Accessed: 24 September 2026).
- Roediger, H.L. and Karpicke, J.D. (2006) 'Test-enhanced learning: taking memory tests improves long-term retention', *Psychological Science*, 17(3), pp. 249–255. doi: 10.1111/j.1467-9280.2006.01693.x.
- Russell, S. and Norvig, P. (2021) *Artificial intelligence: a modern approach*. 4th edn. Hoboken, NJ: Pearson.
- Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson.
- UNESCO World Heritage Centre (no date) *Aapravasi Ghat*. Available at: <https://whc.unesco.org/en/list/1227> (Accessed: 24 September 2026).
- Vite (no date) *Vite: next generation frontend tooling*. Available at: <https://vite.dev> (Accessed: 24 September 2026).
- Vygotsky, L.S. (1978) *Mind in society: the development of higher psychological processes*. Edited by M. Cole, V. John-Steiner, S. Scribner and E. Souberman. Cambridge, MA: Harvard University Press.
- W3C (2018) *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation, 5 June. Available at: <https://www.w3.org/TR/WCAG21/> (Accessed: 24 September 2026).